

Universidad de La Habana
Facultad de Matemática y Computación

Un lenguaje de dominio específico para la generación declarativa de hojas de cálculo de planificación

Autor: Jose Morales Lazo

**Tutor:
MSc. Fernando Rodríguez Flores**

Trabajo de Diploma
presentado en opción al título de
Licenciado en Ciencia de la Computación

La Habana, junio 2026

“La simplicidad es el prerrequisito de la fiabilidad.”

Edsger W. Dijkstra

Agradecimientos

A mi tutor, el MSc. Fernando Rodríguez Flores, por su paciencia, sus ideas y su entusiasmo permanente ante los lenguajes y las hojas de cálculo.

A mi familia, por el apoyo incondicional a lo largo de la carrera.

A los colegas y profesores de MATCOM que convirtieron cada problema académico en una oportunidad de automatizarlo.

Opinión del Tutor

La gestión de la información en entornos académicos genera, de forma recurrente, documentos de planificación tabular: horarios de docencia, calendarios de defensas, distribuciones de aulas. En todos estos casos, la herramienta elegida es la hoja de cálculo. Nada de esto sorprende; lo que sí resulta costoso es que cada vez que se quiere automatizar la creación de esa hoja se repite el mismo patrón: código imperativo que mezcla la lógica del problema con los detalles de la *API* de la biblioteca de salida.

El trabajo que se presenta en este documento aborda precisamente ese costo. Jose Morales Lazo diseñó e implementó un lenguaje de dominio específico (DSL) interno en Common Lisp que permite declarar tablas, columnas calculadas, reglas de visualización condicional y relaciones entre tablas, sin escribir una sola letra de columna Excel, un solo índice de fila ni una cadena de fórmula. El compilador del DSL, implementado como un backend de funciones genéricas CLOS, produce automáticamente el programa Python que construye el archivo `.xlsx` con todas las fórmulas y reglas de formato condicional incrustadas.

El diplomante demostró durante el trabajo una notable capacidad para identificar la abstracción correcta, diseñarla y construirla con rigor. Los tres casos de uso reales validados —horario de defensas de tesis, horario docente de la facultad y horario de programación televisiva— muestran que el lenguaje es suficientemente expresivo para cubrir problemas de complejidad creciente.

Estoy muy satisfecho con los resultados obtenidos y considero que el trabajo reúne los méritos suficientes para optar al título de Licenciado en Ciencia de la Computación.

MSc. Fernando Rodríguez Flores
Facultad de Matemática y Computación
Universidad de La Habana
Junio, 2026

Resumen

La creación automática de hojas de cálculo de planificación exige, en la práctica, código imperativo que entremezcla la lógica del problema —quién tiene conflicto de horario, cuántas defensas acumula un profesor— con los detalles mecánicos de la API de salida: letras de columna Excel, índices de fila absolutos, cadenas de fórmula, llamadas a la biblioteca de formato condicional. Esta mezcla eleva el costo de desarrollo y dificulta la reutilización en problemas similares.

El objetivo de este trabajo es separar ambas capas mediante un lenguaje de dominio específico (DSL) interno en Common Lisp. El programador declara *qué* debe contener el libro —tablas, columnas calculadas, cuantificadores existenciales, reglas de coloreado, totales relativos— y un compilador basado en el Sistema de Objetos de Common Lisp (CLOS) traduce esa declaración a un programa Python que construye el archivo `.xlsx`. Las reglas de formato condicional se incrustan como `FormulaRule` y se reevalúan automáticamente en Excel al editar los datos, sin necesidad de volver a ejecutar el DSL.

La validación sobre tres casos reales de la Facultad de Matemática y Computación de la Universidad de La Habana confirma que el lenguaje captura los patrones recurrentes de la planificación tabular con una cantidad de código significativamente menor que la alternativa imperativa.

Palabras clave: Lenguaje de dominio específico; Hoja de cálculo; Generación automática de código; Common Lisp; CLOS; Macros; Formato condicional; openpyxl.

Abstract

Automatically generating planning spreadsheets requires, in practice, imperative code that interleaves domain logic—who has a scheduling conflict, how many thesis defenses a professor is involved in—with the mechanical details of the output API: Excel column letters, absolute row indices, formula strings, and conditional formatting library calls. This interleaving raises development costs and hampers reuse across similar problems.

The objective of this work is to separate these two layers through an internal domain-specific language (DSL) built in Common Lisp. The programmer declares *what* the workbook should contain—tables, computed columns, existential quantifiers, coloring rules, relative totals—and a compiler based on the Common Lisp Object System (CLOS) translates that declaration into a Python program that builds the `.xlsx` file. Conditional formatting rules are embedded as `FormulaRule` objects and are automatically re-evaluated in Excel when data is edited, without re-running the DSL.

Validation on three real use cases from the Faculty of Mathematics and Computer Science of the University of Havana confirms that the language captures the recurring patterns of tabular planning with significantly less code than the imperative alternative.

Keywords: Domain-specific language; Spreadsheet; Automatic code generation; Common Lisp; CLOS; Macros; Conditional formatting; openpyxl.

Índice general

Agradecimientos	2
Opinión del Tutor	3
Resumen	4
Abstract	5
Introducción	8
1. Preliminares teóricos	11
1.1. Hojas de cálculo	11
1.2. Planificación tabular en el ámbito académico	12
1.3. Lenguajes de dominio específico	12
1.4. Árboles de sintaxis abstracta	13
1.5. El lenguaje de programación Common Lisp	14
1.5.1. Notación prefija y homoiconicidad	14
1.5.2. Macros	14
1.5.3. El Sistema de Objetos de Common Lisp (CLOS)	14
1.5.4. La macro <code>defclass*</code>	15
2. Análisis del dominio de planificación tabular	17
2.1. Casos de uso	17
2.1.1. Horario de defensas de tesis	17
2.1.2. Horario docente de la facultad	18
2.1.3. Horario de programación televisiva	18
2.2. Elementos comunes	19
2.2.1. Tablas relacionadas y referencias cruzadas	19
2.2.2. Columnas calculadas	19
2.2.3. Visualización condicional	20
2.2.4. Posicionamiento relativo	20
2.3. Requisitos del lenguaje	20
3. El lenguaje de dominio específico	22
3.1. Visión general	22
3.2. Árbol de sintaxis abstracta	23
3.2.1. Estructura del workbook	23

3.2.2.	Expresiones de celda	23
3.2.3.	Cuantificación existencial	23
3.2.4.	Formato condicional y posicionamiento	23
3.3.	Definición de tablas: <code>def-table</code>	24
3.4.	Expresiones de celda	27
3.5.	El cuantificador existencial <code>exists</code>	28
3.5.1.	Variante same-table: <code>:all-rows</code>	28
3.5.2.	Variante cross-table: <code>:rows-of</code>	28
3.6.	Formato condicional declarativo	29
3.7.	Ensamblaje del workbook	29
3.8.	Posicionamiento relativo: <code>nav</code>	30
3.9.	Generación y ejecución	30
3.10.	Ejemplo completo: Horario de Defensas de Tesis	30
4.	Generación de código	33
4.1.	Arquitectura del backend	33
4.2.	Resolución de coordenadas	33
4.2.1.	Conversión de número de columna a letra Excel	33
4.2.2.	Construcción del mapa de columnas	34
4.2.3.	Cálculo de rangos de fila	34
4.3.	La función genérica <code>compile-excel-formula</code>	35
4.3.1.	Métodos para expresiones simples	35
4.3.2.	Métodos para rangos y agregados	36
4.4.	Generación de formato condicional	36
4.4.1.	Variante same-table: SUMPRODUCT con exclusión de fila actual	37
4.4.2.	Variante cross-table: SUMPRODUCT con COUNTIFS	37
4.5.	Generación de referencias cruzadas entre hojas	38
4.6.	Generación del programa Python	39
5.	Detalles de implementación	41
5.1.	Organización en cuatro archivos	41
5.2.	Macros del DSL	41
5.2.1.	Macros de expresión	41
5.2.2.	Macros de estructura	43
5.2.3.	El macro <code>nav</code> y el sistema de posicionamiento	44
5.3.	Extensibilidad del sistema	44
5.4.	Validación sobre los casos de uso	45
5.4.1.	Horario de Defensas de Tesis	45
5.4.2.	Horario Docente de la Facultad	45
5.4.3.	Discusión comparativa	46
	Conclusiones	47
	Recomendaciones	49

Introducción

Contexto

La gestión de la información académica en una facultad universitaria produce, de manera recurrente, un tipo particular de documento: la hoja de cálculo de planificación. Los horarios de docencia, los calendarios de defensas de tesis, los cuadros de distribución de aulas y las programaciones de actividades extracurriculares comparten una estructura común: tablas relacionadas, con columnas que resumen información de otras tablas mediante fórmulas, y con celdas que se colorean automáticamente cuando se detecta un conflicto o una condición relevante [15].

La hoja de cálculo es la herramienta elegida para estos documentos por razones prácticas: está disponible en cualquier puesto de trabajo, no requiere instalación de software adicional y produce un resultado que el usuario final puede editar directamente. Sin embargo, cuando el documento debe generarse automáticamente a partir de datos externos —una base de datos, un formulario, un archivo JSON— la simplicidad operativa de la hoja de cálculo se convierte en complejidad de programación.

El problema

El problema central que este trabajo aborda es la mezcla de responsabilidades en el código que genera hojas de cálculo complejas.

Para ilustrarlo, consideremos un caso concreto: construir automáticamente un horario de defensas de tesis en Excel. El documento debe incluir una tabla de defensas con las columnas de rol del tribunal, una tabla de profesores con el número de defensas en que participa cada uno, y una regla de formato condicional que coloree en rojo a los profesores que aparecen en dos defensas programadas en el mismo slot horario.

Un programa imperativo que genera este documento en Python usando la biblioteca `openpyxl` debe conocer simultáneamente:

- que la columna `tutor` ocupa la letra B y `secretario` ocupa la letra F en la hoja de cálculo concreta;
- que los datos de defensas empiezan en la fila 4 y terminan en la fila 6;
- que la fórmula de conteo es `COUNTIF($B$4:$F$6,K4)`;
- que la regla de coloreado condicional es una `FormulaRule` con la expresión `SUMPRODUCT(($B$4:$B$6=$K4)+...) * COUNTIFS(...) > 0 > 0`.

Si el número de defensas cambia, o si se agrega una columna de rol, o si las tablas se reorganizan, *todos* esos valores deben recalcularse manualmente. El código no expresa la intención —“colorear a los profesores con conflicto de horario”— sino el mecanismo concreto de su implementación en una versión específica de la hoja.

Esta situación no es exclusiva del horario de defensas. El horario docente de la facultad, que distribuye grupos, turnos y aulas en múltiples hojas con referencias cruzadas, y el horario de programación televisiva, que calcula el fin de cada programa a partir de su hora de inicio y duración, presentan el mismo patrón: la lógica del problema queda ahogada entre detalles de la API.

La pregunta de investigación central de este trabajo es:

¿Puede un lenguaje de dominio específico separar la intención semántica del mecanismo de implementación, permitiendo generar hojas de cálculo complejas de planificación con un código declarativo de nivel de abstracción superior?

Hipótesis de la investigación

La utilización de un lenguaje de dominio específico (DSL) interno, implementado mediante macros de Common Lisp y un compilador basado en el Sistema de Objetos de Common Lisp (CLOS), permite separar la descripción semántica de una hoja de cálculo de planificación —su estructura tabular, sus relaciones, sus fórmulas y sus condiciones de coloreado— de los detalles concretos de la API de generación de archivos `.xlsx`. Esta separación reduce el código necesario, facilita la reutilización ante cambios en los datos y hace que el programa resultante sea legible por el experto del dominio sin conocimiento de la API de destino.

Objetivo general

Diseñar e implementar un lenguaje de dominio específico en Common Lisp para la declaración y generación automática de hojas de cálculo de planificación, incluyendo tablas relacionadas, columnas calculadas, cuantificación existencial y formato condicional persistente en el archivo `.xlsx`.

Objetivos específicos

1. Analizar las características recurrentes de las hojas de cálculo de planificación en el contexto académico, identificando los patrones comunes de estructura, cálculo y visualización condicional.
2. Diseñar la sintaxis y la semántica de un DSL declarativo capaz de expresar tablas relacionadas, referencias cruzadas entre tablas, columnas computadas, cuantificación existencial y posicionamiento relativo de expresiones auxiliares.
3. Definir un árbol de sintaxis abstracta (AST) cuyas clases representen los conceptos del dominio de manera independiente del backend de generación.

4. Implementar el DSL como un conjunto de macros de Common Lisp que construyen nodos del AST en tiempo de expansión de macros.
5. Implementar un backend de generación de código basado en una función genérica CLOS que traduzca cada tipo de nodo AST a instrucciones Python para la biblioteca `openpyxl`.
6. Validar el lenguaje sobre tres casos de uso reales de la Facultad de Matemática y Computación de la Universidad de La Habana: el horario de defensas de tesis, el horario docente de la facultad y el horario de programación televisiva.

Estructura del documento

El documento se organiza de la siguiente manera.

El capítulo 1 sienta las bases teóricas y conceptuales del trabajo. Se presentan las definiciones fundamentales sobre hojas de cálculo, se introduce el concepto de lenguaje de dominio específico y su uso en la construcción de compiladores, se explica el papel de los árboles de sintaxis abstracta como representación intermedia, y se describen las características de Common Lisp —macros, CLOS y despacho múltiple— que hacen de él una plataforma idónea para el diseño del DSL.

El capítulo 2 analiza el dominio de la planificación tabular a partir de tres casos reales. Se identifican los elementos comunes en estos documentos y se formulan los requisitos concretos que el lenguaje debe satisfacer.

El capítulo 3 describe el lenguaje de dominio específico diseñado. Se presentan el árbol de sintaxis abstracta con sus cuatro familias de nodos, las formas del DSL visible al programador y su semántica, y el flujo completo desde la declaración hasta la producción del archivo `.xlsx`.

El capítulo 4 detalla el proceso de generación de código. Se explica cómo el backend resuelve las coordenadas Excel, cómo compila cada tipo de expresión a cadenas de fórmula, y cómo genera las fórmulas SUMPRODUCT que implementan el formato condicional persistente.

El capítulo 5 describe los detalles de implementación: la organización en cuatro archivos con responsabilidades separadas, la macro `defclass*` que automatiza la definición de nodos del AST, y los principios de extensibilidad del sistema.

El documento concluye con las conclusiones del trabajo, las recomendaciones para investigaciones futuras y las referencias bibliográficas consultadas.

Capítulo 1

Preliminares teóricos

En este capítulo se presentan los conceptos y herramientas que fundamentan el trabajo. La sección 1.1 define los términos clave del dominio de las hojas de cálculo. La sección 1.2 sitúa ese dominio en el contexto académico. Las secciones 1.3 y 1.4 introducen los lenguajes de dominio específico y los árboles de sintaxis abstracta. Finalmente, la sección 1.5 describe las características de Common Lisp que lo convierten en la plataforma de implementación elegida.

1.1 Hojas de cálculo

Una **hoja de cálculo** (*spreadsheet*) es una aplicación para organizar, calcular y visualizar datos en una cuadrícula bidimensional de celdas. Cada celda puede contener un valor literal —un número, una fecha o una cadena de texto— o una **fórmula**: una expresión cuyo resultado se recalcula automáticamente cuando cambia cualquier celda de la que depende [14].

Workbook (libro de trabajo). Un archivo `.xlsx` que agrupa una o más hojas de trabajo. Es la unidad de intercambio del formato Office Open XML [8].

Hoja de trabajo. Una cuadrícula de celdas dentro del workbook, identificada por un nombre (p. ej., **Defensas**).

Celda. La unidad mínima de almacenamiento, referenciada por una dirección del tipo B4 (columna B, fila 4) o **\$B\$4** (referencia absoluta).

Fórmula. Una expresión que comienza con `=` y puede contener referencias a otras celdas, rangos y funciones predefinidas como **COUNTIF**, **COUNTA**, **SUM**, **IF** o **SUMPRODUCT**.

Rango. Un bloque rectangular de celdas contiguas, expresado como **\$B\$4:\$F\$6** (columnas B–F, filas 4–6).

Formato condicional. Un mecanismo que aplica estilos visuales —color de relleno, color de texto, borde— a las celdas de un rango cuando se cumple una condición evaluada en tiempo de apertura o edición del archivo. Una **FormulaRule** es el tipo más general de regla de formato condicional: la condición es una fórmula arbitraria, lo que permite expresar condiciones que involucren otras hojas, otras celdas o funciones de agregación [7].

La biblioteca **openpyxl** es la herramienta Python elegida para producir archivos **.xlsx** en este trabajo. Permite crear workbooks, escribir valores y fórmulas en celdas, y definir **FormulaRule** que quedan incrustadas en el archivo y se evalúan en Excel o LibreOffice Calc sin necesidad de código adicional [7].

1.2 Planificación tabular en el ámbito académico

La planificación tabular agrupa los problemas de organización cuya solución se representa de forma natural como una o más tablas relacionadas. En el contexto académico, los ejemplos más frecuentes son:

- **Horarios de docencia.** Distribución de grupos, asignaturas, profesores y aulas en slots de día y hora a lo largo de la semana. Cada columna calcula la ocupación de un aula a partir de las asignaciones de los distintos grupos.
- **Planificación de defensas de tesis.** Asignación de estudiantes, fechas, horarios y miembros de tribunal (tutor, oponente, presidente, vocal, secretario) a defensas individuales. El requisito crítico es detectar conflictos: un mismo profesor en dos defensas del mismo slot horario.
- **Asignación de docencia.** Relación entre profesores, asignaturas y grupos, con conteos de horas acumuladas y alertas cuando se supera un umbral.

Estos documentos comparten una estructura típica [15]:

1. Dos o más tablas relacionadas que comparten columnas de clave (p. ej., nombre de profesor, slot horario).
2. Columnas calculadas en una tabla que agregan datos de otra (conteos, sumas, búsquedas).
3. Reglas de formato condicional que colorean filas cuando se detecta una condición problemática (conflicto de horario, desequilibrio de carga).
4. Totales y etiquetas fuera de las tablas, cuya posición depende del número de filas de datos, que cambia en cada ejecución.

Estos cuatro patrones son los que el DSL debe capturar de forma declarativa.

1.3 Lenguajes de dominio específico

Un **lenguaje de dominio específico** (*Domain-Specific Language*, DSL) es un lenguaje de programación o especificación diseñado para expresar soluciones dentro de un dominio de problemas concreto, en contraposición a los lenguajes de propósito general que aspiran a cubrir cualquier problema [1].

Definición 1.1 (DSL, [9]). *Un lenguaje de dominio específico es un lenguaje de programación o especificación ejecutable que ofrece, mediante abstracciones apropiadas, poder expresivo centrado en y normalmente restringido a un dominio de problemas particular.*

Los DSL se clasifican en dos grandes tipos según su relación con el lenguaje anfitrión [1]:

DSL externo. Tiene su propia sintaxis independiente, requiere un lexer y un parser propios, y suele estar embebido como datos en otro programa (p. ej., SQL, HTML, $\text{\LaTeX}$).

DSL interno (embebido). Se expresa directamente en el lenguaje anfitrión mediante una API o un sistema de macros que produce código válido del anfitrión [10]. El programador usa el lenguaje anfitrión como plataforma; el DSL es una biblioteca con vocabulario de dominio.

El DSL desarrollado en este trabajo es **interno**: está escrito en Common Lisp, sus formas son expresiones Lisp válidas y no requiere ningún parser externo. Los macros de Lisp operan como el compilador del DSL, expandiendo las formas del dominio en constructores de nodos del AST.

La ventaja principal de los DSL internos en Lisp es que el lenguaje anfitrión es homocónico: el código tiene la misma estructura que los datos (listas anidadas), lo que hace que escribir transformaciones de código —es decir, macros— sea tan natural como escribir funciones ordinarias [4].

1.4 Árboles de sintaxis abstracta

Un **árbol de sintaxis abstracta** (AST, del inglés *Abstract Syntax Tree*) es la representación intermedia estándar en los compiladores e intérpretes. Representa la estructura sintáctica de un programa mediante un árbol cuyos nodos corresponden a los constructores del lenguaje y cuyos hijos representan los subcomponentes de cada constructor [2].

El AST es *abstracto* en el sentido de que omite los detalles de la sintaxis concreta —paréntesis, palabras clave, signos de puntuación— y conserva únicamente la estructura lógica del programa [18].

En el diseño del DSL presentado en este trabajo, el AST cumple el papel de **representación intermedia independiente del backend**: el programador construye el AST mediante las formas del DSL, y el backend lo recorre para producir el código Python de generación del `.xlsx`. Gracias a esta separación, el mismo AST podría en principio alimentar backends alternativos (LibreOffice Calc, CSV, HTML) sin modificar la capa del DSL.

Cada nodo del AST es una instancia de una clase CLOS definida con la macro `defclass*`. Los nodos se dividen en cuatro familias:

1. Nodos de estructura del workbook (`xl-workbook`, `xl-sheet`, `xl-region`, `xl-table`).
2. Nodos de expresión de celda (`xl-expr-column-ref`, `xl-table-range`, `xl-expr-countif`, `xl-expr-if`, etc.).
3. Nodos de cuantificación existencial (`xl-expr-exists`, `xl-domain-rows`).
4. Nodos de formato condicional y posicionamiento (`xl-style-rule`, `xl-anchor`, `xl-nav`, `xl-sheet-fixed-expr`).

Esta organización se describe en detalle en el capítulo 3.

1.5 El lenguaje de programación Common Lisp

Common Lisp es un lenguaje de programación de propósito general creado originalmente por John McCarthy en 1958 [11] y estandarizado como Common Lisp en 1994 [3]. Su elección como plataforma de implementación del DSL se justifica por cuatro características decisivas.

1.5.1 Notación prefija y homoiconicidad

En Common Lisp, todas las expresiones son listas anidadas con notación prefija: el primer elemento es el operador y los siguientes son los argumentos. Una llamada a función, la definición de una clase, una iteración y una expresión del DSL tienen exactamente la misma forma sintáctica.

Esta propiedad, llamada **homoiconicidad**, hace que el código Lisp sea indistinguible de los datos Lisp [4]. El compilador puede inspeccionar y transformar el código del mismo modo que inspecciona y transforma listas de datos. Los DSL internos se benefician directamente de esto: las formas del DSL son listas ordinarias que los macros transforman en código válido.

1.5.2 Macros

Un **macro** de Lisp es una función que opera sobre el código fuente del programa —representado como listas— y devuelve código nuevo que sustituye a la forma original antes de la compilación o evaluación [5].

La diferencia con una función ordinaria es temporal: una función se evalúa en tiempo de ejecución; un macro se expande en tiempo de compilación. Esta distinción permite a los macros introducir nueva sintaxis y nuevas abstracciones sin coste en tiempo de ejecución.

En este trabajo, cada constructo del DSL —`def-table`, `conditional-rendering`, `exists`, `nav`— se implementa como un macro que construye uno o más nodos del AST. El programador escribe en vocabulario del dominio; el macro transforma esa escritura en llamadas a los constructores CLOS.

1.5.3 El Sistema de Objetos de Common Lisp (CLOS)

El **Sistema de Objetos de Common Lisp** (CLOS, del inglés *Common Lisp Object System*) es el sistema de orientación a objetos integrado en el estándar del lenguaje [12]. Sus características principales son:

Herencia múltiple. Una clase puede heredar de varias superclases simultáneamente. Los conflictos de herencia se resuelven mediante el *Class Precedence List* (CPL), un orden lineal determinista [6].

Despacho múltiple. Los métodos no pertenecen a una clase específica. Una función genérica puede despachar sobre los tipos de varios argumentos simultáneamen-

te [13]. Esto significa que el comportamiento de (`compile-excel-formula` `nodo` ...) depende del tipo de `nodo`, no de un método ligado a una clase particular.

Funciones genéricas. Las funciones polimórficas se declaran con `defgeneric` y se especializan con `defmethod`. Añadir soporte para un nuevo tipo de nodo AST sólo requiere añadir un nuevo `defmethod`; el resto del sistema no se modifica.

En este trabajo, CLOS cumple dos funciones. Primero, las clases CLOS representan los nodos del AST: cada tipo de nodo es una clase con slots que almacenan sus atributos. Segundo, la función genérica `compile-excel-formula` usa el despacho múltiple para compilar cada tipo de nodo a su cadena de fórmula Excel correspondiente, sin ninguna estructura `cond` o `case` centralizada.

1.5.4 La macro `defclass*`

Para reducir el código repetitivo de definir clases CLOS y sus constructores, el sistema define la macro `defclass*`, que genera simultáneamente:

- Una clase CLOS con nombre `clase-xl-nombre`, donde cada slot tiene un accessor y un `initarg` del mismo nombre.
- Una función constructora `xl-nombre` con argumentos por clave (`&key`), que crea una instancia de la clase con los valores proporcionados.

Listing 1.1: Implementación de `defclass*`.

```
(defmacro defclass* (class-name parents slots
                    &key (ctr-args-type '&key))
  `(progn
    (defclass ,(sybm 'clase- class-name) ,parents
      ,(loop for s in slots
        collecting (expansion-del-slot s)))
    ,(make-constructor class-name slots
      :argument-type ctr-args-type)))
```

Por ejemplo, la definición del nodo `xl-expr-if`:

Listing 1.2: Ejemplo de uso de `defclass*`.

```
(defclass* xl-expr-if () (test then else))
;; genera:
;; (defclass clase-xl-expr-if () (test :accessor test :
;;   initarg :test)
;;                                     (then :accessor then :
;;   initarg :then)
;;                                     (else :accessor else :
;;   initarg :else))
;; (defun xl-expr-if (&key test then else)
;;   (make-instance 'clase-xl-expr-if :test test :then then
;;     :else else))
```

Este patrón elimina el código repetitivo de definición de nodos y garantiza que cada

clase tiene una función constructora consistente con su interfaz de inicialización.

Capítulo 2

Análisis del dominio de planificación tabular

Para diseñar un lenguaje que capture los patrones recurrentes de la planificación tabular, es necesario estudiar primero los casos concretos que el lenguaje debe resolver. En este capítulo se presentan tres casos de uso reales de la Facultad de Matemática y Computación de la Universidad de La Habana, se identifican los elementos comunes entre ellos y se derivan los requisitos del lenguaje.

2.1 Casos de uso

2.1.1 Horario de defensas de tesis

El proceso de planificación de defensas de tesis de diploma asigna a cada estudiante un conjunto de cinco profesores que forman el tribunal —tutor, oponente, presidente, vocal y secretario—, una fecha y hora de exposición, y un local donde esta se celebra. El coordinador de defensas debe detectar conflictos: un mismo profesor asignado a dos defensas con el mismo slot horario (mismo día y hora), lo que hace imposible su presencia en ambas.

La hoja de cálculo que da soporte a este proceso contiene dos tablas:

Tabla de defensas. Una fila por defensa, con columnas para el estudiante y los cinco roles del tribunal, más las columnas de día, hora y local. Esta tabla no necesita columnas calculadas: toda la información se introduce manualmente.

Tabla de profesores. Una fila por profesor, con su nombre, grado académico y el número de defensas en que participa. Este último dato se calcula automáticamente contando cuántas veces aparece el nombre del profesor en cualquiera de los cinco roles de la tabla de defensas. Además, las filas de los profesores con conflicto de horario se resaltan en rojo.

Los datos del caso de validación incluyen tres defensas y seis profesores. Dos de las defensas comparten el slot **Lunes 10:00** con integrantes de tribunal en común, lo que genera un conflicto que la hoja debe detectar y resaltar:

Las defensas de Juan Díaz y Luis Mora comparten el slot **Lunes 10:00**. El profesor

Cuadro 2.1: Datos de las defensas del caso de validación.

Estudiante	Tutor	Opon.	Pres.	Vocal	Secr.	Día	Hora
Juan Díaz	Piad	García	López	Torres	Ruiz	Lunes	10:00
María Vega	García	López	Piad	Soto	Torres	Lunes	14:00
Luis Mora	López	Torres	García	Ruiz	Soto	Lunes	10:00

López participa como oponente en la primera y como tutor en la segunda; el profesor Torres como vocal en la primera y como oponente en la segunda. Ambos tienen conflicto y deben aparecer resaltados en rojo en la tabla de profesores.

Este caso es el que se usa como ejemplo motivador a lo largo del capítulo 3.

2.1.2 Horario docente de la facultad

La asignación de docencia en la Facultad de Matemática y Computación implica distribuir los turnos de clase de cada semana entre los grupos de las tres carreras (Ciencia de la Computación, Matemática y Ciencia de Datos), en cuatro años cada una. Cada grupo tiene una hoja propia en el workbook con su horario semanal.

La hoja de cálculo de planificación contiene, para cada grupo, tres tablas dispuestas en la misma fila de la hoja:

Tabla de turnos (turno-table). Seis columnas (turno, lunes, martes, miércoles, jueves y viernes), con dos filas Excel por turno lógico (`cell-height = 2`): la primera subfila almacena el nombre de la asignatura y la segunda el aula asignada.

Tabla de estadísticas (stats-table). Una fila por asignatura, con su abreviatura, nombre completo, frecuencia semanal requerida, asignaciones ya realizadas (calculadas contando las apariciones de la abreviatura en la tabla de turnos) y horas que faltan por asignar (calculadas como la diferencia entre frecuencia y asignadas).

Tabla de aulas (aulas-table). Lista de aulas disponibles para el grupo.

Además de las hojas de grupo, existe una hoja de ocupación de aulas que agrega, para cada turno de cada día, los grupos que ocupan cada aula. Esta tabla usa referencias cruzadas dinámicas (`collect-over`) que iteran sobre todas las hojas de grupo y extraen el aula asignada en cada turno. Diecisiete grupos generan diecisiete hojas y una hoja de aulas con referencias a las diecisiete.

Este caso pone a prueba dos características del DSL que el caso de defensas no ejercita: las tablas con celdas de altura compuesta (`cell-height` mayor que 1) y las referencias cruzadas entre hojas del mismo workbook mediante `collect-over`.

2.1.3 Horario de programación televisiva

El tercer caso de uso es el horario de programación semanal de un canal de televisión. Cada programa tiene una hora de inicio, una duración en minutos, un nombre, un tipo y un público objetivo. La hoja de cálculo muestra una página por día, con columnas para

cada uno de esos campos más una columna de hora de fin, calculada automáticamente a partir de la hora de inicio y la duración.

La fórmula de la hora de fin involucra operaciones de tiempo:

$$\text{fin} = \text{TEXT}(\text{TIMEVALUE}(\text{inicio}) + \text{duración}/1440, "hh:mm")$$

donde 1440 es el número de minutos en un día. Además, cada fila se colorea según el tipo de programa (informativo, musical, infantil, etc.) mediante un color distinto en la celda.

Este caso valida que el DSL puede expresar operaciones aritméticas sobre valores de tiempo (`time-add`) y fórmulas de celda simples sin cuantificadores existenciales.

2.2 Elementos comunes

Del análisis de los tres casos se identifican cuatro patrones que se repiten en todos o en la mayoría de los documentos de planificación tabular:

2.2.1 Tablas relacionadas y referencias cruzadas

En todos los casos, el documento contiene más de una tabla cuyos datos están relacionados. La tabla de profesores referencia la tabla de defensas; la tabla de estadísticas referencia la tabla de turnos; la hoja de aulas referencia las hojas de cada grupo.

Estas referencias son de dos tipos:

- **Intrarregión:** ambas tablas están en la misma hoja y se accede a ellas mediante rangos absolutos de columnas (p. ej., `$B$4:$F$6` para los cinco roles de la tabla de defensas).
- **Interhoja:** la tabla referenciada está en otra hoja del mismo workbook (p. ej., la hoja de aulas referencia las hojas de grupos).

El lenguaje debe resolver estas referencias sin que el programador escriba letras de columna ni nombres de hoja: los identifica por los símbolos DSL de las columnas y por el identificador de la tabla.

2.2.2 Columnas calculadas

En todos los casos, al menos una columna de alguna tabla muestra un valor calculado a partir de datos de otra tabla. Los tipos de cálculo más frecuentes son:

COUNTIF. Cuenta cuántas celdas de un rango cumplen un criterio. Ejemplo: número de veces que aparece el nombre de un profesor en los cinco roles de la tabla de defensas.

COUNTA. Cuenta celdas no vacías en un rango. Ejemplo: número de defensas registradas.

SUM. Suma un rango. Ejemplo: total de defensas acumuladas por todos los profesores.

Aritmética de tiempo. Suma una duración en minutos a una hora de inicio. Ejemplo: hora de fin de un programa televisivo.

collect-over. Agrega resultados de la misma posición en múltiples hojas. Ejemplo: grupos que ocupan el Aula 1 en el turno 3 del lunes.

2.2.3 Visualización condicional

En los casos de defensas y de docencia, la condición de coloreado depende de datos de otra tabla. El color no es estático: si el usuario edita los datos directamente en Excel, el coloreado debe recalcularse automáticamente.

El mecanismo adecuado es, por tanto, una **FormulaRule** de formato condicional incrustada en el archivo **.xlsx**, no un coloreado estático aplicado en tiempo de generación. La condición de la regla es una fórmula **SUMPRODUCT** que captura la semántica del cuantificador existencial del DSL.

La forma en que el DSL expresa esta condición —mediante **exists**— y la forma en que el backend la implementa —mediante **SUMPRODUCT**— se detallan en los capítulos 3 y 4, respectivamente.

2.2.4 Posicionamiento relativo

Todos los casos requieren expresiones fuera de las tablas: totales, etiquetas, celdas de resumen. La posición de estas expresiones depende de cuántas filas de datos tiene la tabla, que varía en cada ejecución.

La solución adoptada en el DSL es el sistema de anclas y navegación (**nav**, **ultima-fila**, **primera-fila**): se declara que una expresión va “una fila por debajo de la última fila de datos de la columna **estudiante** de la tabla de defensas”, y el compilador resuelve esa declaración a la coordenada Excel concreta en tiempo de generación.

2.3 Requisitos del lenguaje

Del análisis anterior se derivan los requisitos que el DSL debe satisfacer:

1. **Declaración de tablas.** El lenguaje debe permitir definir tablas con nombre, con una lista de columnas identificadas por símbolos, y con soporte para celdas de altura y anchura compuesta.
2. **Columnas computadas.** Debe ser posible asociar una expresión del DSL a una columna; el compilador genera la fórmula Excel correspondiente para cada fila de datos.
3. **Referencias intrarregión.** El lenguaje debe resolver referencias a columnas de la misma tabla (**col**) y a rangos de columnas de otra tabla en la misma región (**trange**, **tcol**).
4. **Cuantificación existencial.** El lenguaje debe ofrecer una forma de expresar condiciones de existencia sobre filas de la misma tabla (**:all-rows**) o de otra tabla (**:rows-of**), especificando columnas de coincidencia y columnas de solapamiento.
5. **Formato condicional persistente.** Las reglas de coloreado condicional deben traducirse a **FormulaRule** en el **.xlsx**, de modo que persistan y se reevalúen cuando el usuario edita el archivo.

6. **Posicionamiento relativo.** Debe ser posible colocar expresiones fijas en posiciones calculadas como un ancla más pasos direccionales, sin coordenadas absolutas codificadas.
7. **Referencias interhoja.** El lenguaje debe soportar referencias dinámicas a celdas de otras hojas mediante iteración sobre una lista de nombres de hojas (`collect-over`).
8. **Ensamblaje declarativo.** El workbook completo debe poderse describir con formas del DSL (`libro`, `hoja`, `tabla`) sin ningún conocimiento de las coordenadas del archivo de salida.
9. **Independencia del backend.** El AST no debe contener ningún detalle del target de generación. Agregar un nuevo tipo de expresión no debe requerir modificar el DSL; solo añadir un nuevo método al backend.

La tabla 2.2 muestra cómo cada requisito se manifiesta en los casos de uso analizados.

Cuadro 2.2: Requisitos del lenguaje y casos de uso en que se manifiestan.

Requisito	Defensas	Facultad	TV
Declaración de tablas	✓	✓	✓
Columnas computadas	✓	✓	✓
Referencias intrarregión	✓	✓	—
Cuantificación existencial	✓	—	—
Formato condicional persistente	✓	—	—
Posicionamiento relativo	✓	✓	—
Referencias interhoja	—	✓	—
Ensamblaje declarativo	✓	✓	✓
Independencia del backend	✓	✓	✓

Capítulo 3

El lenguaje de dominio específico

3.1 Visión general

La generación de libros de cálculo de planificación implica, habitualmente, escribir código imperativo que mezcla la lógica del problema con los detalles de la API de hojas de cálculo: letras de columna, índices de fila, cadenas de fórmula Excel y llamadas a una biblioteca de bajo nivel. El objetivo del lenguaje de dominio específico (DSL) desarrollado en este trabajo es eliminar esa mezcla, permitiendo al programador declarar *qué* debe contener el libro —tablas, columnas calculadas, reglas de coloreado condicional— sin ocuparse de *cómo* se produce el archivo.

El DSL está implementado como un conjunto de macros de Common Lisp que construyen, en tiempo de expansión, un *árbol de sintaxis abstracta* (AST) cuyos nodos son instancias de clases CLOS. La canalización de compilación comprende tres etapas claramente separadas:

1. **Expansión de macros.** Cada forma del DSL (p. ej. `def-table`, `exists`, `conditional-rendering`) se expande en una llamada al constructor CLOS del nodo AST correspondiente. El resultado es un árbol de objetos en memoria.
2. **Generación de código.** El árbol se recorre por el backend, que traduce cada nodo a cadenas de fórmula Excel (expresiones de celda y reglas de formato condicional) y a instrucciones de construcción del libro. El backend es el único punto que conoce los nombres de columna reales, los índices de fila y las cadenas de la API de destino.
3. **Ejecución.** El programa generado se ejecuta, produciendo el archivo `.xlsx` con los datos, fórmulas y reglas de formato condicional incrustadas.

Un principio central de diseño es la *separación entre intención e implementación*: el DSL expresa la semántica del problema (“colorear los profesores que tienen un conflicto de horario”) y el backend decide el mecanismo concreto (“generar una `FormulaRule` `SUMPRODUCT` que persiste y recalcula en Excel”). El programador del DSL no escribe ninguna fórmula Excel.

3.2 Árbol de sintaxis abstracta

El árbol de sintaxis abstracta (AST) es la representación intermedia que separa la capa del DSL de la capa del generador de código. Cada concepto del problema —una tabla, una expresión de celda, un cuantificador existencial— queda capturado como un nodo AST con atributos propios. El backend opera únicamente sobre este árbol; no interpreta texto ni inspecciona las formas del DSL.

Todos los nodos del AST se definen con la macro `defclass*`, descrita en la sección 1.5.4, que genera simultáneamente una clase CLOS `class-xl-nombre` y una función constructora `xl-nombre` con parámetros por clave.

Listing 3.1: Patrón de definición de nodo AST con `defclass*`.

```
;; defclass* genera la clase y el constructor xl-expr-exists:
(defclass* xl-expr-exists ()
  (bind-var domain match-keys overlap-cols self-in-
    cols))

;; El constructor acepta los slots como :keywords:
(xl-expr-exists :bind-var      'r
                :domain        (xl-domain-rows :table-id nil)
                :match-keys     '(dia hora)
                :overlap-cols   '(tutor oponente presidente
                                   vocal secretario)
                :self-in-cols   nil)
```

Los nodos del AST se agrupan en cuatro categorías según su papel en el árbol.

3.2.1 Estructura del workbook

Los nodos de estructura representan los contenedores del libro de cálculo. Son los nodos de más alto nivel del árbol: el workbook agrupa hojas, las hojas agrupan regiones y las regiones agrupan tablas.

3.2.2 Expresiones de celda

Las expresiones de celda son los nodos que el backend traduce a cadenas de fórmula Excel. Abarcan tanto referencias a columnas de la misma tabla o de otra como operaciones de hoja de cálculo concretas y literales de texto.

3.2.3 Cuantificación existencial

La cuantificación existencial es el mecanismo del DSL para expresar condiciones de la forma “existe alguna fila que cumpla ciertos criterios”. Mantenerla como un nodo AST propio, y no como una expresión booleana explícita, permite al backend generar la comprobación más adecuada según el contexto: una fórmula `SUMPRODUCT` vectorial en el caso de Excel, u otra implementación para un backend diferente.

3.2.4 Formato condicional y posicionamiento

Cuadro 3.1: Nodos de estructura del workbook.

Nodo	Atributos	Papel
<code>xl-workbook</code>	<code>name</code> , <code>sheets</code>	Raíz del árbol. <code>name</code> es el nombre del archivo <code>.xlsx</code> ; <code>sheets</code> es la lista de hojas.
<code>xl-sheet</code>	<code>name</code> , <code>regions</code> , <code>fixed-expressions</code>	Una hoja del libro. Contiene regiones horizontales y expresiones en posición relativa fuera de las tablas.
<code>xl-region</code>	<code>tables</code>	Agrupar tablas colocadas horizontalmente en la misma hoja, separadas por una columna en blanco.
<code>xl-table</code>	<code>id</code> , <code>col-names</code> , <code>headers</code> , <code>contenido</code> , <code>computed</code> , <code>style-rules</code> , <code>cell-height</code> , <code>cell-width</code>	Plantilla de tabla. <code>col-names</code> enumera los símbolos DSL de columna; <code>computed</code> asocia columnas a expresiones de fórmula; <code>style-rules</code> almacena las reglas de coloreado condicional.

3.3 Definición de tablas: `def-table`

La forma central del DSL para definir datos tabulares es `def-table`. Produce una *plantilla* de tabla reutilizable: una descripción abstracta de las columnas, sus cabeceras, las fórmulas que se calculan a partir de ellas y la regla de coloreado que se aplica sobre sus filas. La plantilla no contiene datos; se instancia con datos concretos al ensamblar el workbook.

Listing 3.2: Sintaxis completa de `def-table`.

```
(def-table nombre-tabla
  ((col1 "Cabecera1") (col2 "Cabecera2") ...) ; columnas
  :cell-height N      ; filas Excel por fila logica (default 1)
  :cell-width N       ; columnas Excel por columna logica (
    default 1)
  :first-row N        ; fila Excel donde empiezan los datos (
    default 4)
  :inst-params (p1 p2 ...) ; parametros de instancia (
    opcional)
  :computed           ; formulas derivadas (opcional)
  ((colN (expresion-dsl)))
  :render             ; regla de coloreado (opcional)
  (conditional-rendering
    :condition (expresion-booleana)
    :target-columns (col1 col2 ...)))
```

Cuadro 3.2: Nodos de expresión de celda (selección).

Nodo	Atributos	Papel
<code>xl-expr-column-ref</code>	<code>name</code> , <code>context</code>	Referencia a la columna <code>name</code> en la fila actual. Produce la letra Excel de esa columna más el número de fila relativo.
<code>xl-table-range</code>	<code>table-id</code> , <code>from-col</code> , <code>to-col</code>	Rango absoluto en una tabla específica de la región (referencia cruzada).
<code>xl-expr-countif</code>	<code>count-range</code> , <code>criteria</code>	Fórmula <code>COUNTIF</code> sobre un rango con criterio.
<code>xl-expr-counta</code>	<code>count-range</code>	Fórmula <code>COUNTA</code> : cuenta celdas no vacías.
<code>xl-expr-sum</code>	<code>count-range</code>	Fórmula <code>SUM</code> sobre un rango.
<code>xl-expr-if</code>	<code>test</code> , <code>then</code> , <code>else</code>	Condicional <code>IF</code> de tres ramas.
<code>xl-expr-equals</code>	<code>a</code> , <code>b</code>	Igualdad entre dos expresiones.
<code>xl-expr-and</code> , <code>xl-expr-or</code>	<code>a</code> , <code>b</code>	Conjunción y disyunción lógica.
<code>xl-expr-concat</code>	<code>a</code> , <code>b</code>	Concatenación de texto (<code>&</code>).
<code>xl-expr-string</code>	<code>value</code>	Literal de texto; produce una cadena fija en la celda.
<code>xl-expr-time-add</code>	<code>a</code> , <code>b</code>	Suma de hora más duración en minutos.
<code>xl-expr-collect-over</code>	<code>groups</code> , <code>sheet-var</code> , <code>body</code>	Agregación sobre múltiples hojas del workbook.

Cuadro 3.3: Nodos de cuantificación existencial.

Nodo	Atributos	Papel
<code>xl-expr-exists</code>	<code>bind-var</code> , <code>match-keys</code> , <code>overlap-cols</code> , <code>self-in-cols</code>	Cuantificador existencial estructurado. El backend decide el mecanismo de comprobación según el tipo de dominio.
<code>xl-domain-rows</code>	<code>table-id</code>	Dominio de filas. <code>table-id = nil</code> indica la tabla actual; un símbolo identifica una tabla específica de la región.

La macro `def-table` genera una función con nombre `nombre-tabla` que acepta `:data` (lista de filas), `:params` (parámetros de instancia con valores fijos) y los `:inst-params`

Cuadro 3.4: Nodos de formato condicional y posicionamiento relativo.

Nodo	Atributos	Papel
<code>xl-style-rule</code>	<code>rule-condition</code> , <code>target-columns</code>	Regla de coloreado condicional. <code>rule-condition</code> es un nodo de expresión booleana; <code>target-columns</code> enumera las columnas que se colorean.
<code>xl-anchor</code>	<code>table-id</code> , <code>col-name</code> , <code>anchor-type</code>	Punto de referencia dentro de una tabla. <code>anchor-type</code> es <code>:ultima-fila</code> o <code>:primera-fila</code> .
<code>xl-nav</code>	<code>anchor</code> , <code>steps</code>	Posición calculada como ancla más secuencia de pasos direccionales.
<code>xl-sheet-fixed-expos</code> , <code>expr</code>		Expresión colocada en una posición relativa de la hoja. <code>pos</code> es un <code>xl-nav</code> ; <code>expr</code> es el nodo de expresión.

declarados, y devuelve un nodo `xl-table` con todos los slots rellenos.

El listado siguiente define la tabla de profesores del caso de defensas:

Listing 3.3: Tabla de profesores con columna computed y regla de coloreado.

```
(def-table profesores-table
  ((nombre "") (grado "") (defensas ""))
  :cell-height 1
  :cell-width 1
  :computed
    ;; Cuenta cuantas veces aparece este profesor en los
    ;; cinco roles
    ((defensas (countif (trange defensas-table tutor
                                secretario)
                        (col nombre))))
  :render
    (conditional-rendering
     :condition
       (exists (r :rows-of defensas-table)
        :self-in (tutor oponente presidente vocal
                    secretario)
        :matching (dia hora))
     :target-columns (nombre)))
```

La variable `r` en la forma `exists` es un alias documental de la fila iterada, análogo a un alias de tabla en SQL. La regla de coloreado declara que la celda `nombre` debe resaltarse cuando el profesor participa en alguna defensa con conflicto de horario; el

mecanismo lo decide el backend.

3.4 Expresiones de celda

El DSL ofrece un conjunto de formas simbólicas para construir expresiones de celda sin escribir cadenas de fórmula Excel directamente.

Cuadro 3.5: Formas de expresión del DSL y su traducción Excel.

Forma DSL	Significado / traducción Excel
(col nombre)	Valor de la columna nombre en la fila actual. Se resuelve a la letra Excel más el número de fila relativo (p. ej., B4).
(tcol tabla nombre)	Valor de la columna nombre en la tabla tabla de la misma región. Evita ambigüedad cuando dos tablas comparten nombres de columna.
(trange tabla desde)	Rango de la columna desde de tabla , todas las filas de datos. P. ej., \$B\$4:\$B\$6.
(trange tabla desde hasta)	Rango desde desde hasta hasta de tabla . P. ej., \$B\$4:\$F\$6.
(countif rango criterio)	Fórmula COUNTIF(rango, criterio).
(counta rango)	Fórmula COUNTA(rango): celdas no vacías.
(sum-range rango)	Fórmula SUM(rango).
(str "texto")	Literal de texto.
(equals a b)	Igualdad: produce a=b en la fórmula.
(_and a b), (_or a b)	AND(a,b), OR(a,b).
(concat a b)	Concatenación de texto: (a&b).
(time-add hora minutos)	TEXT(TIMEVALUE(hora)+(minutos/1440),"hh:mm").
(_if cond entonces sino)	IF(cond, entonces, sino).
(add a b), (subtract a b)	Operaciones aritméticas: (a+b), (a-b).
(collect-over lista (var) expr)	Agrega expr sobre cada hoja de lista , ligando var al nombre de la hoja actual.

Listing 3.4: Expresiones DSL y sus fórmulas Excel generadas.

```
;; (col nombre) en profesores-table (columna K), fila 4
(col nombre)                ;; -> K4

;; Rango de las cinco columnas de rol de defensas-table
(trange defensas-table tutor secretario) ;; -> $B$4:$F$6

;; La formula COUNTIF resultante
(countif (trange defensas-table tutor secretario) (col nombre
))
```

```
;; -> COUNTIF($B$4:$F$6, K4)

;; Hora de fin de un programa televisivo
(time-add (col inicio) (col duracion))
;; -> TEXT(TIMEVALUE(A4)+(B4/1440), "hh:mm")
```

3.5 El cuantificador existencial **exists**

El cuantificador **exists** declara condiciones de existencia sobre filas de una tabla. La motivación es que preguntas del tipo “existe otra defensa con el mismo horario” son muy frecuentes en problemas de planificación, pero expresarlas mediante disyunciones y bucles explícitos produce código verboso y difícil de leer.

El cuantificador existe en dos variantes según el dominio de búsqueda.

3.5.1 Variante **same-table**: **:all-rows**

Busca otra fila dentro de la *misma* tabla que comparta valores en columnas de coincidencia y tenga al menos un valor común en columnas de solapamiento. Se usa para detectar si dos filas del propio conjunto entran en conflicto.

Listing 3.5: Cuantificador **exists** sobre la tabla actual.

```
(exists (r :all-rows)
  :matching (dia hora) ; mismos valores en ambas
  filas
  :any-overlap (tutor oponente presidente vocal secretario))
  ; algun rol compartido
```

La semántica es: “existe otra fila de esta tabla (distinta de la actual) cuyas columnas **dia** y **hora** coinciden con las de la fila actual, y que además comparte al menos un valor en las columnas de rol”.

3.5.2 Variante **cross-table**: **:rows-of**

Busca en las filas de *otra* tabla, relacionando el valor de la celda actual con columnas de esa tabla.

Listing 3.6: Cuantificador **exists** sobre otra tabla (cross-table).

```
(exists (r :rows-of defensas-table)
  :self-in (tutor oponente presidente vocal secretario)
  ; el valor actual aparece en alguno de estos
  roles
  :matching (dia hora))
  ; ese slot tiene mas de una defensa programada
```

La semántica es: “existe alguna fila de **defensas-table** en la que el valor de la celda actual (el nombre del profesor) aparece en una de las cinco columnas de rol, y cuyo par (**dia**, **hora**) tiene más de una defensa programada”.

En ambas variantes el alias de fila es puramente documental. El backend genera directamente una fórmula vectorial SUMPRODUCT sin evaluar el símbolo como variable de programa. Su papel es análogo al de un alias de tabla en SQL.

3.6 Formato condicional declarativo

La forma `conditional-rendering` permite declarar el coloreado condicional de celdas sin escribir fórmulas Excel ni llamadas a la API de formato.

Listing 3.7: Sintaxis de conditional-rendering.

```
(conditional-rendering
  :condition      (expresion-booleana)
  :target-columns (col1 col2 ...))
```

Cuando el nodo condición es una instancia de `xl-expr-exists`, el backend no produce colores estáticos en tiempo de compilación, sino una `FormulaRule` de Excel incrustada en el archivo `.xlsx`. Esta regla persiste en el libro y se reevalua automáticamente cada vez que el usuario edita los datos directamente en Excel.

La decisión de usar `FormulaRule` en lugar de colorear celdas directamente es exclusiva del backend: el programador del DSL no indica en ningún lugar qué mecanismo de formato se utilizará. Las fórmulas concretas generadas se detallan en el capítulo 4.

3.7 Ensamblaje del workbook

Una vez definidas las plantillas de tabla, el programador ensambla el workbook con tres macros: `libro` crea el nodo raíz del AST; `hoja` agrupa tablas y expresiones fijas en una pestaña del libro; `tabla` instancia una plantilla con datos concretos.

Listing 3.8: Ensamblaje del workbook con libro, hoja y tabla.

```
(libro nombre-libro
  :filename "Archivo.xlsx"
  :hojas (list
    (hoja "NombreHoja"
      (tabla plantilla1 :data *DATOS-1*)
      (tabla plantilla2 :data *DATOS-2*)
      :fixed-expressions
        (((nav ...) (expresion))
         ((nav ...) (expresion))))))
```

Cuando una `hoja` recibe más de una `tabla`, el generador las coloca en la misma *región horizontal*: las tablas se disponen de izquierda a derecha separadas por una columna en blanco. Las referencias cruzadas (`trange`) se resuelven dentro de la misma región.

La macro `hoja-v` coloca cada tabla en su propia región vertical, apilando las tablas de arriba hacia abajo en lugar de de izquierda a derecha.

3.8 Posicionamiento relativo: `nav`

La forma `nav` calcula la posición de una celda como un *ancla* fija más una secuencia de pasos direccionales. Las anclas disponibles son `ultima-fila` y `primera-fila`, que identifican la última o primera fila de datos de una columna de una tabla.

Listing 3.9: Sintaxis de `nav` y anclas de tabla.

```
;; Una fila debajo de la ultima fila de datos
(nav (ultima-fila tabla-id col-name) abajo 1)

;; Una fila debajo y dos columnas a la derecha
(nav (ultima-fila tabla-id col-name) abajo 1 derecha 2)

;; Una fila encima de la primera fila de datos
(nav (primera-fila tabla-id col-name) arriba 1)
```

El compilador resuelve `ultima-fila` a la fila Excel concreta en tiempo de generación y produce una referencia de celda absoluta que apunta al lugar correcto independientemente del número de filas de datos.

3.9 Generación y ejecución

Las dos últimas formas del DSL cierran el ciclo de compilación: `xl-generate` recorre el árbol AST y produce el programa Python de construcción del libro; `xl-run-generated` ejecuta ese programa y crea el archivo `.xlsx`.

Listing 3.10: Compilación del AST y ejecución.

```
;; Recorre el AST y genera el programa Python
(xl-generate horario-tesis "horario-tesis.py")

;; Ejecuta el programa generado y crea el archivo .xlsx
(xl-run-generated "horario-tesis.py")
```

3.10 Ejemplo completo: Horario de Defensas de Tesis

El listado siguiente es el programa DSL completo del caso de validación principal. Contiene todas las construcciones descritas en este capítulo: definición de tablas, columna `computed`, cuantificador `exists` cross-table, formato condicional declarativo, ensamblaje del workbook y posicionamiento relativo con `nav`.

Listing 3.11: Programa DSL completo: Horario de Defensas de Tesis.

```
(load "dsl-directo.lisp")
(load "data-tutorial.lisp") ; *DATOS-DEFENSAS*, *DATOS-
                             PROFESORES*

;; -- TABLA 1: defensas
-----
```

```

(def-table defensas-table
  ((estudiante "") (tutor "") (oponente "") (presidente "")
   (vocal "") (secretario "") (dia "") (hora "") (local ""))
  :cell-height 1
  :cell-width 1)

;; -- TABLA 2: profesores
-----
(def-table profesores-table
  ((nombre "") (grado "") (defensas ""))
  :cell-height 1
  :cell-width 1
  :computed
  ((defensas (countif (trange defensas-table tutor
                               secretario)
                      (col nombre))))
  :render
  (conditional-rendering
   :condition
   (exists (r :rows-of defensas-table)
    :self-in (tutor oponente presidente vocal
              secretario)
    :matching (dia hora))
   :target-columns (nombre)))

;; -- WORKBOOK
-----
(libro horario-tesis
 :filename "Horario_Tesis.xlsx"
 :hojas (list
  (hoja "Defensas"
   (tabla defensas-table :data *DATOS-DEFENSAS*)
   (tabla profesores-table :data *DATOS-PROFESORES*)
   :fixed-expressions
   (((nav (ultima-fila defensas-table estudiante) abajo
           1)
    (str "Total defensas:"))
    ((nav (ultima-fila defensas-table estudiante) abajo
           1 derecha 1)
     (counta (trange defensas-table estudiante)))
    ((nav (ultima-fila profesores-table nombre) abajo 1)
     (str "Total profesores:"))
    ((nav (ultima-fila profesores-table nombre) abajo 1
           derecha 2)
     (sum-range (trange profesores-table defensas)))))))

(xl-generate horario-tesis "horario-tesis.py")
(xl-run-generated "horario-tesis.py")

```

El programa genera Horario_Tesis.xlsx con la estructura siguiente:

- Columnas A–I: **defensas-table** con tres filas de datos.
- Columna J: separador en blanco.
- Columnas K–M: **profesores-table** con seis filas de datos y la columna **Defensas** calculada mediante **COUNTIF**.
- Fila de totales debajo de cada tabla.
- **FormulaRule** aplicada al rango de la columna **nombre**: los profesores con conflicto de horario aparecen resaltados en rojo.

Capítulo 4

Generación de código

La generación de código es la etapa del sistema que traduce el AST construido por las formas del DSL a un programa Python que produce el archivo `.xlsx`. Este capítulo describe cómo el backend resuelve las coordenadas Excel, cómo compila cada tipo de nodo de expresión a una cadena de fórmula, y cómo genera las fórmulas SUMPRODUCT que implementan el formato condicional persistente.

4.1 Arquitectura del backend

El backend está organizado en dos archivos con responsabilidades separadas:

code-generation-utils.lisp. Funciones auxiliares, variables dinámicas de configuración y la función de entrada principal. Incluye: `col->letter` (conversión de número de columna a letra Excel), `build-col-map` (construcción del mapa símbolo DSL $\rightarrow$ letra), `xl-generate` y `xl-run-generated`.

generate-code-direct.lisp. Los métodos CLOS de la función genérica `compile-excel-formula` y los métodos del generador de código Python. Este archivo es el único lugar del sistema donde pueden aparecer letras de columna Excel, cadenas de fórmula y llamadas a `openpyxl`.

El principio de separación de responsabilidades queda codificado en la propia arquitectura: `ast-def.lisp` declara *qué existe*, `dsl-directo.lisp` declara *cómo se escribe* y `generate-code-direct.lisp` declara *a qué se traduce*.

4.2 Resolución de coordenadas

4.2.1 Conversión de número de columna a letra Excel

La función `col->letter` convierte un entero n (la posición ordinal de una columna, con base 1) a la letra o letras de columna correspondientes en Excel: columna 1 $\rightarrow$ A, columna 26 $\rightarrow$ Z, columna 27 $\rightarrow$ AA, etc.

Listing 4.1: Función `col->letter`.

```
(defun col->letter (n)
  (with-output-to-string (s)
```

```
(loop while (> n 0)
  do (multiple-value-bind (q r) (floor (1- n) 26)
      (setf n q)
      (write-char (code-char (+ 65 r)) s))))
```

El algoritmo es una conversión de base 10 a base 26 con el ajuste habitual ($\lfloor (n-1)/26 \rfloor$, $((n-1) \bmod 26) + 65$) que mapea los residuos a los caracteres ASCII de las letras mayúsculas.

4.2.2 Construcción del mapa de columnas

Para cada tabla de la región se construye un *mapa de columnas*: una lista de asociación que relaciona cada símbolo DSL de columna con su letra Excel absoluta. La función `build-col-map` construye el mapa de una tabla sola, asumiendo que empieza en la columna 1. La función `build-table-col-maps` construye el mapa global de toda la región, teniendo en cuenta el desplazamiento de cada tabla dentro de la región:

Listing 4.2: Construcción del mapa de columnas de una región.

```
(defun build-col-map (tbl)
  (loop for name in (col-names tbl)
    for i from 1
    collect (cons name (col->letter i))))

(defun build-table-col-maps (tables offsets)
  (loop for tbl in tables
    for offset in offsets
    collect (cons (id tbl)
      (loop for name in (col-names tbl)
        for i from 1
        collect (cons name
          (col->letter (+
            offset i))))))))
```

El resultado se almacena en la variable dinámica `*table-col-maps*`, disponible durante toda la compilación de la región.

4.2.3 Cálculo de rangos de fila

Cada tabla tiene una primera fila de datos (`first-row`, configurable, por defecto 4) y una última fila que depende del número de filas de datos y de `cell-height`:

$$last-row = first-row + |datos| \times cell-height - 1 \quad (4.1)$$

Estos valores se almacenan en la variable dinámica `*table-data-rows*` como una lista de asociación (`table-id . (first-row . last-row)`), lo que permite que los nodos `xl-table-range` usen los rangos de fila correctos para cada tabla aunque la compilación se realice dentro del contexto de una tabla diferente.

4.3 La función genérica `compile-excel-formula`

El método central del backend es la función genérica `compile-excel-formula`, que toma un nodo AST y devuelve la cadena de fórmula Excel correspondiente. Su firma es:

Listing 4.3: Declaración de `compile-excel-formula`.

```
(defgeneric compile-excel-formula
  (expr col-map data-names row-num first-row last-row)
  (:documentation
    "Compila un nodo AST de expresion a string de formula
    Excel"))
```

Los parámetros son:

expr El nodo AST a compilar.

col-map El mapa de columnas de la tabla actual (`sym → letra`).

data-names Lista de símbolos DSL de la tabla actual (para `VLOOKUP`).

row-num Número de fila Excel que se está compilando (para referencias relativas).

first-row Primera fila de datos de la tabla actual.

last-row Última fila de datos de la tabla actual.

El despacho múltiple de CLOS garantiza que, para cada tipo de nodo, se ejecuta el método correcto sin ningún `cond` centralizado. Añadir un nuevo tipo de expresión al lenguaje sólo requiere:

1. Definir una nueva clase con `defclass*` en `ast-def.lisp`.
2. Definir un macro en `dsl-directo.lisp` que construya el nodo.
3. Definir un `defmethod` de `compile-excel-formula` en `generate-code-direct.lisp`.

Ningún código existente necesita modificarse.

4.3.1 Métodos para expresiones simples

Listing 4.4: Métodos `compile-excel-formula` para expresiones simples.

```
;; Referencia a columna de la tabla actual: "K4"
(defmethod compile-excel-formula
  ((e clase-xl-expr-column-ref) col-map data-names
   row-num first-row last-row)
  (let* ((letter (cdr (assoc (name e) col-map)))
        (ctx (context e)))
    (if (typep ctx 'clase-xl-expr-previous-row)
        (format nil "~a~a" letter (1- row-num))
        (format nil "~a~a" letter row-num))))

;; Comparacion de igualdad: "K4=B4"
(defmethod compile-excel-formula
```

```

((e clase-xl-expr-equals) col-map data-names
 row-num first-row last-row)
(format nil "~a=~a"
  (compile-excel-formula (a e) col-map data-names
                        row-num first-row last-row)
  (compile-excel-formula (b e) col-map data-names
                        row-num first-row last-row)
  ))

;; Literal de texto: "\"Total\""
(defmethod compile-excel-formula
  ((e clase-xl-expr-string) col-map data-names
   row-num first-row last-row)
  (format nil "~s" (value e)))

```

4.3.2 Métodos para rangos y agregados

Listing 4.5: Método para xl-table-range.

```

;; Rango absoluto de columnas de una tabla especifica de la
;; region.
;; Usa *table-data-rows* para obtener los limites exactos de
;; esa tabla.
(defmethod compile-excel-formula
  ((e clase-xl-table-range) col-map data-names
   row-num first-row last-row)
  (let* ((tmap (cdr (assoc (table-id e) *table-col-
                           maps*)))
        (from-letter (cdr (assoc (name (from-col e)) tmap)))
        (to-letter (if (to-col e)
                        (cdr (assoc (name (to-col e)) tmap)
                                )
                        from-letter)))
        (tbl-rows (cdr (assoc (table-id e) *table-data-
                               rows*)))
        (act-first (if tbl-rows (car tbl-rows) first-row))
        (act-last (if tbl-rows (cdr tbl-rows) last-row)))
    (format nil "$~a$~a:$~a$~a"
      from-letter act-first to-letter act-last)))

```

La clave técnica de este método es que consulta `*table-data-rows*` con el identificador de la *tabla referenciada*, no con el de la tabla actual. Así, `(trange defensas-table tutor secretario)` compila al rango de `defensas-table` aunque se esté compilando una fórmula de `profesores-table`.

4.4 Generación de formato condicional

El formato condicional es el aspecto técnicamente más elaborado de la generación de código. El backend recibe un nodo `xl-expr-exists` y produce una fórmula SUMPRO-

DUCT que implementa la semántica del cuantificador existencial declarado.

4.4.1 Variante same-table: SUMPRODUCT con exclusión de fila actual

Cuando `exists` usa `:all-rows`, el backend genera una fórmula SUMPRODUCT que, para cada fila, multiplica tres factores:

1. Coincidencia en las columnas de agrupación (`:matching`).
2. Exclusión de la fila actual (para no comparar una fila consigo misma).
3. Solapamiento en al menos una columna de rol (`:any-overlap`).

Para una regla con `:matching` (`dia hora`) y `:any-overlap` (`tutor ... secretario`), la fórmula generada para la fila 4 es:

Listing 4.6: Fórmula Excel CF para la variante same-table (fila 4).

```
=SUMPRODUCT (
  ($G$4:$G$6=$G4) *           -- dia coincide en ambas filas
  ($H$4:$H$6=$H4) *           -- hora coincide en ambas
    filas
  (ROW($B$4:$B$6)<>ROW($B4)) * -- excluir la fila actual
  (($B$4:$B$6=$B4)+($C$4:$C$6=$C4)+...+($F$4:$F$6=$F4))
) > 0
```

Las referencias con columna fija y fila relativa (p.ej. `$G4`) hacen que la fórmula se evalúe correctamente para cada fila del rango: al desplazarse de la fila 4 a la fila 5, las referencias de “fila actual” avanzan automáticamente. El resultado de SUMPRODUCT es el número de filas del dominio que satisfacen las tres condiciones; la comparación `> 0` convierte ese conteo en un valor booleano para la regla de formato condicional.

4.4.2 Variante cross-table: SUMPRODUCT con COUNTIFS

Cuando `exists` usa `:rows-of`, el backend genera una fórmula SUMPRODUCT que combina dos condiciones:

1. Que el valor de la celda actual aparezca en al menos una columna de rol de la tabla referenciada (`:self-in`).
2. Que el slot de tiempo de esa fila tenga más de una defensa programada (`:matching` con COUNTIFS).

Para la regla de `profesores-table` con `:self-in` (`tutor ... secretario`) y `:matching` (`dia hora`), la fórmula generada para la celda K4 es:

Listing 4.7: Fórmula Excel CF para la variante cross-table (celda K4).

```
=SUMPRODUCT (
  (($B$4:$B$6=$K4)+($C$4:$C$6=$K4)+($D$4:$D$6=$K4)+
  ($E$4:$E$6=$K4)+($F$4:$F$6=$K4)) *
  (COUNTIFS($G$4:$G$6,$G$4:$G$6,
    $H$4:$H$6,$H$4:$H$6) > 1)
) > 0
```

La referencia \$K4 corresponde al nombre del profesor en la celda actual (columna K bloqueada, fila relativa). Los rangos \$B\$4:\$B\$6 a \$F\$4:\$F\$6 son las columnas de rol de `defensas-table`, completamente absolutas.

COUNTIFS con cada par de columnas de coincidencia iguales a sí mismas produce el número de filas que comparten exactamente los mismos valores en esas columnas; comparar el resultado con 1 selecciona los slots de tiempo con más de una defensa. El producto SUMPRODUCT de ambas condiciones devuelve el número de filas de `defensas-table` en que el profesor actual participa en un slot conflictivo.

Al ser una `FormulaRule` incrustada en el archivo Excel, cualquier edición posterior de los datos provoca una reevaluación automática del coloreado, sin necesidad de volver a ejecutar el DSL.

4.5 Generación de referencias cruzadas entre hojas

El nodo `xl-expr-collect-over` itera sobre una lista de hojas del workbook y agrega la expresión del cuerpo para cada una. El mecanismo de iteración usa la variable dinámica `*sheet-env*`: durante la compilación de cada término, la variable de iteración (`sheet-var`) se liga temporalmente al nombre de la hoja actual.

Listing 4.8: Método `compile-excel-formula` para `collect-over`.

```
(defmethod compile-excel-formula
  ((e clase-xl-expr-collect-over) col-map data-names
   row-num first-row last-row)
  (let* ((groups (groups e))
        (sh-var (sheet-var e))
        (body (body e))
        (terms (mapcar
                 (lambda (g)
                   ;; Liga sh-var -> g solo durante este
                   termino
                   (let ((*sheet-env* (acons sh-var g *
                                             sheet-env*)))
                     (compile-excel-formula body col-map
                                             data-names
                                             row-num first-
                                             row last-row
                                             )))
                 groups)))
        (inner (format nil "~{~a~&~}" terms)))
    (format nil "SUBSTITUTE(TRIM(~a),\" \",\",\",\\")" inner)))
```

La fórmula resultante concatena los N términos con `&` y los envuelve en `SUBSTITUTE(TRIM(...))` para eliminar espacios extras y reemplazarlos con comas, produciendo una lista de grupos separada por comas.

Para el horario de la facultad con 17 grupos, el método genera una fórmula que agrega la ocupación del Aula 1 en el turno 3 del lunes a través de las 17 hojas del workbook,

sin que el programador haya escrito ningún nombre de hoja ni letra de columna.

4.6 Generación del programa Python

El generador principal recorre el árbol del workbook y emite un programa Python que usa `openpyxl` para construir el archivo `.xlsx`. El programa generado:

1. Crea un workbook `openpyxl`.
2. Para cada hoja, crea una hoja de trabajo y la configura.
3. Para cada tabla de cada región, escribe las cabeceras y los datos.
4. Para cada columna calculada, escribe la fórmula en cada fila de datos (compilada por `compile-excel-formula` en tiempo de generación del Python).
5. Para cada regla de estilo condicional, crea una `FormulaRule` con la fórmula `SUMPRODUCT` generada y la añade al rango correspondiente.
6. Para cada expresión fija, resuelve la posición `nav` y escribe la expresión compilada en la celda correspondiente.
7. Guarda el workbook en el archivo `.xlsx` configurado.

El listado siguiente muestra el fragmento del programa Python generado para la columna calculada `defensas` de `profesores-table`:

Listing 4.9: Fragmento del programa Python generado para la columna `computed`.

```
# Columna: defensas (profesores-table)
ws["M4"] = "=COUNTIF($B$4:$F$6,K4)"
ws["M5"] = "=COUNTIF($B$4:$F$6,K5)"
ws["M6"] = "=COUNTIF($B$4:$F$6,K6)"
ws["M7"] = "=COUNTIF($B$4:$F$6,K7)"
ws["M8"] = "=COUNTIF($B$4:$F$6,K8)"
ws["M9"] = "=COUNTIF($B$4:$F$6,K9)"

# FormulaRule para profesores-table, columna nombre
from openpyxl.formatting.rule import FormulaRule
from openpyxl.styles import PatternFill
red_fill = PatternFill(start_color="FF6666",
                       end_color="FF6666", fill_type="solid")
formula = ("SUMPRODUCT(((B$4:B$6=$K4)+(C$4:C$6=$K4)+("
           "($D$4:$D$6=$K4)+($E$4:$E$6=$K4)+($F$4:$F$6=$K4))*"
           " "
           "(COUNTIFS(G$4:$G$6,$G$4:$G$6,$H$4:$H$6,$H$4:$H$6"
           ")>1))>0)")
ws.conditional_formatting.add(
    "K4:K9",
    FormulaRule(formula=[formula], fill=red_fill))
```

Las fórmulas de las celdas están calculadas en tiempo de generación del Python: el programa Python que se ejecuta sólo asigna cadenas de texto a las celdas. Las referencias

relativas dentro de las fórmulas las evalúa Excel al abrir el archivo. La `FormulaRule`, en cambio, se reevalua en Excel dinámicamente cada vez que cambian los datos.

Capítulo 5

Detalles de implementación

Este capítulo describe los aspectos internos del sistema: la organización de los archivos con sus responsabilidades, los macros que implementan el DSL, el patrón de extensibilidad del backend y la validación del sistema sobre los tres casos de uso presentados en el capítulo 2.

5.1 Organización en cuatro archivos

El sistema está organizado en cuatro archivos Lisp con responsabilidades estrictamente separadas. La dependencia entre ellos es unívoca: cada archivo carga únicamente al que le precede.

Esta separación tiene consecuencias prácticas importantes: es posible cargar `ast-def.lisp` y construir un AST sin haber cargado el backend, lo que facilita las pruebas de la sintaxis y la semántica del DSL de forma independiente.

5.2 Macros del DSL

Los macros del DSL se clasifican en tres grupos según su papel en la jerarquía del AST.

5.2.1 Macros de expresión

Los macros de expresión construyen nodos de la familia de expresiones de celda. Todos son equivalencias directas: reciben los argumentos del programador y los pasan al constructor CLOS correspondiente, haciendo el quoting adecuado para los símbolos del dominio.

Listing 5.1: Macros de expresión representativos.

```
;; (col nombre) -> xl-expr-column-ref con nombre quoteado
(defmacro col (name &optional context)
  (if context
    '(xl-expr-column-ref :name ',name :context ,context)
    '(xl-expr-column-ref :name ',name :context nil)))

;; (trange tabla desde) / (trange tabla desde hasta)
```

Cuadro 5.1: Archivos del sistema y sus responsabilidades.

Archivo	Responsabilidad
<code>codigo-tesis.lisp</code>	Utilidades generales: <code>symb</code> , <code>mkstr</code> , <code>keywd</code> , <code>ppme</code> y la macro <code>defclass*</code> . No conoce ningún concepto del dominio de hojas de cálculo.
<code>ast-def.lisp</code>	Definición de todas las clases del AST con <code>defclass*</code> . Backend-agnóstico: no contiene ninguna letra de columna Excel, cadena de fórmula ni referencia a ninguna biblioteca de salida.
<code>code-generation-utils.lisp</code> <code>generate-code-direct.lisp</code>	Backend Excel/Python. Único lugar del sistema donde viven letras de columna, cadenas de fórmula, estructuras <code>openpyxl</code> y conocimiento de la disposición concreta del workbook. El primero contiene las utilidades y variables dinámicas; el segundo, los métodos <code>compile-excel-formula</code> .
<code>dsl-directo.lisp</code>	Macros del DSL: <code>def-table</code> , <code>tabla</code> , <code>hoja</code> , <code>hoja-v</code> , <code>libro</code> , <code>exists</code> , <code>conditional-rendering</code> , <code>nav</code> , <code>col</code> , <code>tcol</code> , <code>trange</code> , <code>countif</code> , <code>counta</code> , <code>sum-range</code> , <code>str</code> , <code>concat</code> , <code>time-add</code> , y demás. No contiene lógica de generación de código.

```
;; -> xl-table-range con los dos extremos como xl-expr-table-
    col-ref
(defmacro trange (table-id from &optional to)
  '(xl-table-range
    :table-id ',table-id
    :from-col (xl-expr-table-col-ref :table-id ',table-id
                                     :name ',from :context
                                     nil)
    :to-col   ,(if to
                  '(xl-expr-table-col-ref :table-id ',table-id
                                           :name ',to :context nil)
                  nil)))

;; (exists (var :rows-of tabla) :self-in (...) :matching
  (...))
;; -> xl-expr-exists con domain cross-table
(defmacro exists ((var &rest domain-spec)
                  &key matching any-overlap self-in)
  (let ((domain-type (first domain-spec)))
    (ecase domain-type
      (:all-rows
```

```

      '(xl-expr-exists :bind-var      ',var
                      :domain        (xl-domain-rows :table
                                                    -id nil)
                      :match-keys    ',matching
                      :overlap-cols  ',any-overlap
                      :self-in-cols  nil))
    (:rows-of
     (let ((table-id (second domain-spec)))
       '(xl-expr-exists :bind-var      ',var
                       :domain        (xl-domain-rows
                                      :table-id ',table-id)
                       :match-keys    ',matching
                       :overlap-cols  ',any-overlap
                       :self-in-cols  ',self-in))))))

```

La regla de quoting de los macros es determinista: un símbolo que es nombre de columna o tabla en el dominio se quotea (`'`, `sym`); un símbolo que es una variable Lisp en scope no se quotea (`,`, `sym`). Esta distinción es explícita en todos los macros.

5.2.2 Macros de estructura

Los macros `def-table`, `hoja`, `hoja-v` y `libro` construyen los nodos de la jerarquía del workbook.

`def-table` es el más complejo: genera una función que construye un `xl-table` con sus slots rellenos. Los `:inst-params` declarados se añaden a la firma de esa función, lo que permite instanciar la misma plantilla de tabla con valores de parámetro distintos:

Listing 5.2: Expansión simplificada de `def-table`.

```

(def-table aulas-dia-table
  ((dia "") (aula1 "") ...)
  :inst-params (dia)
  :computed ((aula1 (collect-over *grupos-all* (g) ...))))

;; Genera la funcion:
(defun aulas-dia-table (&key data params dia)
  (xl-table :id 'aulas-dia-table
            :col-names '(dia aula1 ...)
            :computed (list (cons 'aula1 (collect-over ...)))
            ...))

;; Que luego se instancia con:
(tabla aulas-dia-table :dia lun :data *datos-aulas-lunes*)
;; -> (aulas-dia-table :data *datos-aulas-lunes* :dia 'lun)

```

5.2.3 El macro `nav` y el sistema de posicionamiento

El macro `nav` convierte una secuencia plana de pares dirección–cantidad en una lista de cons-celdas:

Listing 5.3: Implementación del macro `nav`.

```
(defmacro nav (anchor &rest steps)
  (let ((step-pairs
        (loop for (dir n) on steps by #'cddr
              collect '(cons ',dir ,n))))
    `(xl-nav :anchor ,anchor
             :steps (list ,@step-pairs))))

;; Ejemplo: (nav (ultima-fila t c) abajo 1 derecha 2)
;; Expande a:
;; (xl-nav :anchor (xl-anchor :table-id 't :col-name 'c
;;                             :anchor-type :ultima-fila)
;;         :steps (list (cons 'abajo 1) (cons 'derecha 2)))
```

El backend resuelve los pasos en tiempo de generación: dado el número de fila de la última celda de datos, suma o resta las cantidades según la dirección y produce la coordenada Excel absoluta.

5.3 Extensibilidad del sistema

La extensibilidad es una consecuencia directa del uso de CLOS y de la separación en archivos. Considrese el ejemplo de añadir soporte para la función `AVERAGEIF` de Excel, que actualmente no está en el lenguaje.

Paso 1 — Nuevo nodo AST (en `ast-def.lisp`):

```
(defclass* xl-expr-averageif () (average-range criteria))
```

Paso 2 — Nuevo macro (en `dsl-directo.lisp`):

```
(defmacro averageif (range-expr criteria)
  `(xl-expr-averageif :average-range ,range-expr
                     :criteria ,criteria))
```

Paso 3 — Nuevo método (en `generate-code-direct.lisp`):

```
(defmethod compile-excel-formula
  ((e clase-xl-expr-averageif) col-map data-names
   row-num first-row last-row)
  (let ((range-str (compile-excel-formula
                    (average-range e) col-map data-names
                    row-num first-row last-row))
        (criteria (compile-excel-formula
                  (criteria e) col-map data-names
                  row-num first-row last-row)))
```

```
(format nil "AVERAGEIF(~a,~a)" range-str criteria)))
```

Ningún código existente necesita modificarse. El macro `defclass*` garantiza que la nueva clase tiene exactamente la misma interfaz que las demás.

5.4 Validación sobre los casos de uso

Los tres casos de uso descritos en el capítulo 2 se validaron ejecutando el DSL y verificando manualmente el archivo `.xlsx` generado.

5.4.1 Horario de Defensas de Tesis

El programa DSL del listado 3.11 genera `Horario_Tesis.xlsx` con la estructura esperada. La verificación confirmó:

- La columna `Defensas` de `profesores-table` muestra los valores correctos: Piad = 2, García = 2, López = 3, Torres = 3, Ruiz = 2, Soto = 2.
- Los profesores López y Torres, que participan en las dos defensas del slot `Lunes 10:00`, aparecen resaltados en rojo.
- Los totales en la fila inferior a las tablas son correctos: “Total defensas: 3” y “Total profesores: 6” con las sumas esperadas.
- La `FormulaRule` persiste en el archivo: al cambiar manualmente la hora de la tercera defensa de `10:00` a `14:00`, el resaltado de López y Torres desaparece automáticamente en Excel.

5.4.2 Horario Docente de la Facultad

El programa `ast-facultad.lisp` genera `Horario_Facultad.xlsx` con diecisiete hojas de grupo (Ciencia de la Computación en cuatro años, Matemática en cuatro años y Ciencia de Datos en cuatro años) más una hoja de ocupación de aulas.

La verificación confirmó:

- Las columnas `asignadas` y `faltan` de `stats-table` calculan correctamente para cada grupo a partir del contenido de `turno-table`.
- La hoja de aulas muestra en cada celda los grupos que ocupan cada aula en cada turno, con la lista de grupos separada por comas producida por las fórmulas `SUBSTITUTE(TRIM(...))`.
- Las celdas compuestas de `turno-table` (`cell-height = 2`) se generan correctamente: la subfila superior muestra la asignatura y la inferior el aula.

Este caso ejercita, además, la característica de `inst-params`: la tabla `aulas-dia-table` se define una sola vez y se instancia una vez por día de la semana, con `:dia lun`, `:dia mar`, etc.

5.4.3 Discusión comparativa

Para ilustrar el beneficio del DSL, se estima la cantidad de código necesaria para implementar el caso de defensas de forma imperativa directa en Python con `openpyxl`, en comparación con el programa DSL.

La versión imperativa directa requiere calcular manualmente las letras de columna, construir las cadenas de fórmula `COUNTIF` con interpolación de texto, y ensamblar la fórmula `SUMPRODUCT` con referencias absolutas calculadas a partir del número de filas. Una implementación representativa ocupa aproximadamente 80–100 líneas de Python.

El programa DSL equivalente (listado 3.11) ocupa 40 líneas, de las cuales la mayor parte son la declaración de datos (`def-table`, `libro`) y la expresión de intención (`exists`, `conditional-rendering`). Ninguna línea contiene una letra de columna, un índice de fila o una cadena de fórmula.

Más importante que el número de líneas es que el programa DSL es resistente a cambios estructurales: añadir una columna a `defensas-table` sólo requiere añadir un par (`nombre` "Cabecera") a la lista de columnas; el compilador recalcula automáticamente todas las letras y rangos. En la versión imperativa, el mismo cambio requiere actualizar manualmente todas las referencias a columnas afectadas.

Conclusiones

El trabajo alcanzó los objetivos propuestos. Se diseñó e implementó un lenguaje de dominio específico en Common Lisp para la generación declarativa de hojas de cálculo de planificación, y se validó sobre tres casos reales de la Facultad de Matemática y Computación de la Universidad de La Habana.

Las conclusiones principales son las siguientes.

1. **La separación entre intención e implementación es alcanzable con un DSL interno en Lisp.** El sistema permite declarar tablas relacionadas, columnas calculadas, cuantificadores existenciales y reglas de coloreado condicional sin escribir ninguna letra de columna Excel, ninguna cadena de fórmula ni ninguna llamada a la API de destino. El backend es el único punto del código donde viven esos detalles.
2. **Las macros de Common Lisp son el mecanismo adecuado para implementar el DSL.** La homoiconicidad del lenguaje hace que los macros sean transformadores de código completamente naturales: cada forma del DSL es una lista anidada que el macro convierte en constructores CLOS, sin ningún parser externo.
3. **El despacho múltiple de CLOS ofrece extensibilidad sin modificar el código existente.** Añadir un nuevo tipo de expresión al lenguaje requiere únicamente tres acciones: definir la clase del nodo con `defclass*`, definir el macro de superficie y definir el método del backend. El patrón se replicó consistentemente en los más de veinte tipos de nodo implementados.
4. **El cuantificador `exists` captura correctamente las condiciones de conflicto de planificación.** Las dos variantes (`:all-rows` y `:rows-of`) cubren los patrones de conflicto más frecuentes en los documentos analizados. El backend genera fórmulas SUMPRODUCT que implementan esas semánticas de forma robusta y producen `FormulaRule` persistentes en el `.xlsx`.
5. **El posicionamiento relativo con `nav` elimina las coordenadas absolutas de los programas del DSL.** Totales, etiquetas y celdas de resumen se colocan siempre en la posición correcta independientemente del número de filas de datos, lo que hace al programa resiliente ante cambios en el tamaño de los conjuntos.
6. **El sistema genera archivos `.xlsx` con formato condicional activo.** La decisión de usar `FormulaRule` en lugar de coloreado estático significa que la hoja de cálculo sigue siendo funcional cuando el usuario la edita directamente en Excel,

sin necesidad de volver a ejecutar el DSL.

Recomendaciones

A partir del trabajo realizado se identifican las siguientes líneas de extensión para investigaciones futuras.

1. **Soporte para más funciones de celda.** El DSL actual cubre `COUNTIF`, `COUNTA`, `SUM`, `IF`, `AVERAGEIF` y operaciones de tiempo. Funciones como `INDEX/MATCH`, `XLOOKUP`, `PIVOTBY` y las nuevas funciones de matrices dinámicas de Excel 365 podrían añadirse siguiendo exactamente el mismo patrón de extensión descrito en la sección 5.3.
2. **Backends alternativos.** La independencia del backend respecto al AST hace posible implementar generadores de código para otros formatos: ODS (LibreOffice Calc), CSV anotado, HTML con tablas, o incluso bases de datos SQL. La capa del DSL no requiere ningún cambio.
3. **Interfaz de usuario gráfica.** El AST es una estructura de datos de primera clase en Lisp. Una interfaz gráfica podría visualizar y editar el AST directamente, permitiendo a usuarios no programadores configurar las reglas de coloreado y los cálculos sin escribir código.
4. **Verificación de tipos del DSL.** El sistema actual no comprueba en tiempo de expansión de macros que los símbolos de columna referenciados en `exists` y `trange` existan en las tablas correspondientes. Un verificador de tipos a nivel del AST detectaría estos errores antes de la generación del código Python.
5. **Generación incremental.** El sistema genera el programa Python completo en cada ejecución. Un modo incremental que detecte qué tablas cambiaron y sólo regenere las hojas afectadas reduciría el tiempo de ciclo en workbooks con muchas hojas.
6. **Plantillas reutilizables.** Las plantillas de tabla definidas con `def-table` podrían exportarse a una biblioteca de plantillas estándar para los documentos más comunes de la facultad (horarios de docencia, defensas, actas de exámen), reduciendo el esfuerzo de configuración inicial para cada nuevo caso.

Bibliografía

- [1] M. Fowler. *Domain-Specific Languages*. Addison-Wesley Professional, 2010.
- [2] A. V. Aho, M. S. Lam, R. Sethi y J. D. Ullman. *Compilers: Principles, Techniques, and Tools*. Pearson/Addison-Wesley, 2^a ed., 2006.
- [3] G. L. Steele. *Common Lisp: The Language*. Digital Press, 2^a ed., 1990.
- [4] P. Graham. *On Lisp: Advanced Techniques for Common Lisp*. Prentice-Hall, 1993.
- [5] P. Seibel. *Practical Common Lisp*. Apress, 2005.
- [6] S. E. Keene. *Object-Oriented Programming in Common Lisp: A Programmer's Guide to CLOS*. Addison-Wesley, 1989.
- [7] E. Gazoni y C. Clark. openpyxl — A Python library to read/write Excel 2010 xlsx/xlsm/xltx/xltm files. <https://openpyxl.readthedocs.io>, 2010–2024.
- [8] ISO/IEC. *ISO/IEC 29500: Information technology — Office Open XML File Formats*. International Organization for Standardization, 2008.
- [9] A. van Deursen, P. Klint y J. Visser. Domain-specific languages: an annotated bibliography. *ACM SIGPLAN Notices*, 35(6):26–36, 2000.
- [10] M. Mernik, J. Heering y A. M. Sloane. When and how to develop domain-specific languages. *ACM Computing Surveys*, 37(4):316–344, 2005.
- [11] J. McCarthy. Recursive functions of symbolic expressions and their computation by machine, Part I. *Communications of the ACM*, 3(4):184–195, 1960.
- [12] D. G. Bobrow, L. G. DeMichiel, R. P. Gabriel, S. E. Keene, G. Kiczales y D. A. Moon. Common Lisp Object System specification. *ACM SIGPLAN Notices*, 23(S):1–142, 1988.
- [13] G. Kiczales, J. des Rivières y D. G. Bobrow. *The Art of the Metaobject Protocol*. MIT Press, 1991.
- [14] Microsoft Corporation. *Excel for Microsoft 365 — Reference Guide*. Microsoft Docs, 2023.
- [15] R. A. Cápiro Colomar. *LD MAG: Lenguaje para el diseño de microaplicaciones de gestión*. Trabajo de Diploma, Facultad de Matemática y Computación, Universidad de La Habana, 2025.
- [16] D. A. Watt. *Programming Language Design Concepts*. Wiley, 2004.

-
- [17] B. Liskov y S. Zilles. Programming with abstract data types. *ACM SIGPLAN Notices*, 9(4):50–59, 1974.
 - [18] M. L. Scott. *Programming Language Pragmatics*. Morgan Kaufmann, 4^a ed., 2015.